



Dipartimento di Elettronica e Informazione

Politecnico di Milano

20133 Milano (Italia)
Piazza Leonardo da Vinci, 32
Tel. (+39) 02-2399.3400
Fax (+39) 02-2399.3411

Advanced Computer Architecture

July 21, 2022

Prof. Donatella Sciuto and Marco Santambrogio

Name
Last Name
Professor:

You have 90' to complete the exam

Problem 1 (6pts)	
Problem 2 (6pts)	
Problem 3 (6pts)	
Question 1 (5pts)	
Question 2 (5pts)	
Question 3 (5pts)	
Total (32pts)	

To pass the exam (the following conditions are in AND):

- The overall score has to be equal or higher than 18
- At least 1 problem and 1 Question has to be correct

Problem 1

Consider two CPUs: CPU1 e CPU2. CPU1 has clock cycle of 2 ns while CPU2 has an operating frequency of 700MHz. Given the following frequencies of occurrence of the instructions for the two CPUs

Operation type	Frequency	CPU1	CPU2
A	0.3	2	2
B	0.1	3	3
C	0.2	4	3
D	0.3	2	2
E	0.1	4	3

- a) Compute the average CPI for CPU1 and CPU2
b) Which is the slowest CPU?
- c) Assume that CPU1 is modified in such a way that:
- All instructions require 8 clock cycles;
 - Memory stalls are not considered;
 - The miss penalty is 7 clock cycles

Assuming a miss rate of 23% and assuming 3 memory references on average for each instruction determine the impact on performance of the cache with respect to an ideal cache?

Answer

a)

$$\text{CPI-CPU1} = 0.3 \cdot 2 + 0.1 \cdot 3 + 0.2 \cdot 4 + 0.3 \cdot 2 + 0.1 \cdot 4 = 0.6 + 0.3 + 0.8 + 0.6 + 0.4 = \mathbf{2.7}$$

$$\text{CPI-CPU2} = 0.3 \cdot 2 + 0.1 \cdot 3 + 0.2 \cdot 3 + 0.3 \cdot 2 + 0.1 \cdot 3 = 0.6 + 0.3 + 0.6 + 0.6 + 0.3 = \mathbf{2.4}$$

b) Speedup = Exe-CPU1/Exe-CPU2 =

$$\begin{aligned} &= ((IC * CPI1)/Freq1) * (Freq2/(IC * CPI2)) = \\ &= (IC * CPI1 * Freq2)/(IC * CPI2 * Freq1) = \\ &= (CPI1 * Freq2)/(CPI2 * Freq1) = \\ &= (2.7 * 700\text{MHz})/(2.4 * 500\text{MHz}) = 1890/1200 = \mathbf{1.575} \end{aligned}$$

c)

CPI_exe = 8; MISS Penalty = 7; MISS Rate = 0.23; Memory references = 3

CPI_ideal_cache (MISS Rate = 0) = **8**

$$\begin{aligned} \text{CPI_cache} &= \text{CPI_exe} + (\text{References} * \text{MISSPenalty} * \text{MISSRate}) = \\ &= 8 + (3 * 7 * 0.23) = 8 + 4.83 = \mathbf{12.83} \end{aligned}$$

Problem 2

Consider the following access pattern on a two-processor system with a direct-mapped, write-back cache with one cache block and a two cache block memory. Assume the MESI protocol is used, with write-back caches, write-allocate, and invalidation of other caches on write (instead of updating the value in the other caches).

Time	After Operation	P1 cache block state	P2 cache block state	Memory at block 0 up to date?	Memory at block 1 up to date?
0	P1: read block 0	Exclusive (0)	Invalid (1)	Yes	Yes
1	P2: write block 1	Exclusive (0)	Mod (1)	Yes	No
2	P1: write block 0	Mod (0)	Mod (1)	No	No
3	P2: read block 0	Shared (0)	Shared (0)	Yes	Yes
4	P1: write block 0	Mod (0)	Invalid (0)	No	Yes
5	P2 : write block 1	Mod (0)	Mod (1)	No	No
6	P1: read block 0	Mod (0)	Mod (1)	No	No
7	P2: write block 0	Invalid (0)	Mod (0)	No	Yes
8	P1: read block 0	Shared (0)	Shared (0)	Yes	Yes
9	P2: read block 0	Shared (0)	Shared (0)	Yes	Yes

Problem 3

Assume that the following code has been executed on a CPU with SCOREBOARD.

	Instruction	ISSUE	READ OPERAND	EXE COMPLETE	WB
I1	LD F1, 16 (R1)	1	2	6	7
I2	ADDD F12, F1, F2	2	8	13	14
I3	MULTD F2, F4, F3	3	4	15	16
I4	DIVD F3, F12, F5	4	15	26	27
I5	SD F3, 4 (R1)	5	28	32	33
I6	SUBD F2, F12, F4	17	18	23	24

A. List all the possible conflicts in the code.

B. Is there a "configuration" that can respect the shown execution?
How many units? Which kind? What latency?

C. If the previous table was not correct, please, write the right one and specify the number, kind and latency for each unit.

Answer 3.A

RAW F1 I1-I2
RAW F12 I2-I4
RAW F12 I2-I6
RAW F3 I4-I5

WAW F2 I3-I6

WAR F2 I2-I3
WAR F2 I2-I6
WAR F3 I3-I4

Answer 3.B

A possible configuration consists of:

- 2 MUL/DIV units and 11 CC of latency
- 1 ADDD/SUBD unit and 5 CC of latency
- 2 Memory Unit and 4 CC of latency
- 1 write port is enough

Other solutions are possible.

Answer 3.C

Considering the previous answers, no answer is needed here.

Question 1

The following 5 questions are True/False (you do not have to motivate/support your answers).

Each of them will count 1 point

Question 1.1

A scalar architecture can let instructions from different threads to be executed at the same clock cycle.

Given the previous statement, confirm if it is TRUE or FALSE and **effectively support** your answer.

Circle the **right** answer: True False

Answer 1.1

FALSE! A scalar architecture is a single issue architecture.

Question 1.2

The ARM big.LITTLE architecture, considering the fact that is a multicore architecture where the cores are sharing the same ISA, cannot be considered a heterogenous architecture.

Circle the **right** answer: True False

Answer 1.2

FALSE! The (at least) two cores are different, therefore, it's an heterogenous architecture even if they are sharing the same ISA.

Question 1.3

A physically distributed memory can be used to implement a shared-memory architecture

Circle the **right** answer: True False

Answer 1.3

TRUE! Both the centralized memory and the distributed memory models can be used to implement a shared-memory architecture

Question 1.4

A VLIW Architecture has to have multiple Program Counters to load the necessary Multiple Data

Circle the **right** answer: True False

Answer 1.4

FALSE! A VLIW architecture is a SINGLE instruction architecture (the bundle is a SINGLE Very Long Instruction), therefore we need just one Program Counter

Question 1.5

Increasing the number of stages in a pipeline, it is always improving the performance

Circle the **right** answer: True False

Answer 1.5

FALSE! Increasing the number of stages in a pipeline may lead to a higher number of overlapping situations but also to a much more complex control unit with may not improve, in the end, the performance of the architecture.

Question 2

Explain the difference among Dependencies and Hazards.

Question 3

Explain the main differences between loop unrolling and software pipelining?

Answer 3

Software pipelining can be thought of as "symbolic" loop unrolling: the main difference is that loop unrolling needs to add some startup code before the loop as well as finish up code at the end.

On the other hand, loop unrolling needs to replicate the entire code of the loop.

For what concern the register pressure, the amount of registers used are not related to the type of scheduling but only to the architecture used.

Due to the fact that loop unrolling at each iteration needs to repeat the computation, it needs to pay the cost of overhead and wind-down more times than software pipelining.

For what concerns the power consumption, because of the fact that they are both static scheduling techniques, the power consumption is lower wrt dynamic scheduling techniques. Hence, power consumption does not depend on the type of scheduling technique used.